

# CAHIER DES CHARGES FONCTIONNEL ET TECHNIQUE

**Projet :** "SecurePost" - Système de Numérisation Sécurisé de Main Courante et Gestion de Commissariat

**Cahier des Charges :** Projet Étudiant / Cadre Académique

**Version :** 2.0 (Spécifications Full-Stack, PostgreSQL & Traçabilité Métier)

## 1. Présentation générale et Objectif du Projet

L'objectif de ce projet est de concevoir et développer une application web Full-Stack, responsive et hautement sécurisée, baptisée "**SecurePost**". Cet outil est destiné à moderniser et numériser la gestion quotidienne d'un petit commissariat de police.

L'application remplace les registres papier traditionnels par un système numérique fiable permettant d'enregistrer et de suivre trois types d'actes fondamentaux :

- **Les Mains Courantes** : Déclarations de faits sans qualification pénale initiale, consignées par les agents de permanence.
- **Les Plaintes** : Signalements formalisés d'infractions (délits ou crimes) nécessitant une suite judiciaire.
- **Les Procès-Verbaux (PV) d'audition** : Transcriptions officielles des déclarations des victimes, témoins ou suspects.

L'accent est mis sur la **traçabilité absolue des actions** (audit log pour éviter toute altération ou suppression frauduleuse), la fluidité sur le terrain et la génération automatique de statistiques de criminalité pour appuyer les décisions du commandement local.

## 2. Acteurs du Projet (MOA & MOE)

Pour ancrer ce projet dans un cadre professionnel et réaliste, la gouvernance est définie comme suit :

- **Maîtrise d'Ouvrage (MOA)** : Mr Webster L'architecte, seigneur du Temple de Cartenon. Il agit en tant que client final (représentant l'administration policière), exprime les besoins fonctionnels, valide les étapes clés, s'assure de la conformité légale et sécuritaire du produit et facilite le double éveil du codeur.
- **Maîtrise d'Œuvre (MOE)** : Les étudiants. La MOE est responsable de la conception technique, de la modélisation de la base de données sécurisée, du développement du code (Frontend et Backend), du respect des critères de confidentialité (RGPD / secrets d'enquête) et du déploiement de la solution .

## 3. Exigences de la MOA (Sécurité & Contraintes Techniques)

La MOA impose le respect strict des critères de performance, d'intégrité et de réalisme suivants :

- **Authenticité et Réalisme des Données (Mock Data)** : Les données de démonstration

injectées initialement dans le système (Seed) doivent refléter le quotidien réel d'un commissariat local (ex: Arrondissements de Pointe-Noire comme le Centre-ville, Tié-Tié, Mpaka). Le tableau d'objets doit contenir au moins 15 à 20 entrées réalistes de mains courantes, plaintes et PV fictifs (ex: "Vol à l'arraché au grand marché", "Différend de voisinage à Lumumba", "Perte de documents officiels").

- **Architecture Technologique (Contraintes strictes) :**
  - **Frontend** : Léger, rapide et épuré. Aucun framework lourd (comme React ou Angular) n'est autorisé pour cette phase afin de garantir la maîtrise absolue des langages de base (HTML5, CSS3/Tailwind, JavaScript Vanilla).
  - **Backend** : Développement d'une API REST rigoureuse (ex: Node.js, PHP OO, Python) gérant les droits d'accès, la journalisation des actions et le traitement des données.
  - **Base de données** : Utilisation obligatoire de **PostgreSQL**. La structure des tables doit appliquer des contraintes d'intégrité fortes (clés primaires, étrangères) pour empêcher toute perte de données.
- **Code Couleur et Disponibilité Visuelle** : Le statut et le niveau de gravité d'une affaire doivent être immédiatement identifiables sur l'interface:
  - *Plaintes / Affaires* : Enquête en cours (Bleu) | Transmis au procureur (Orange) | Classé sans suite (Gris) .
  - *Niveau d'urgence / Gravité* : Faible (Gris) | Moyen (Orange) | Critique / Crime (Rouge)

## 4. Spécifications du Backend, Base de Données et Traçabilité

### 4.1. Modélisation de la Base de Données (PostgreSQL)

La base de données doit refléter l'organisation du commissariat via plusieurs concepts clés (Workspaces administratifs, Utilisateurs, Actes, Logs) :

- **Workspaces / Brigades** : ex: "Brigade Anti-Criminalité", "Sûreté Urbaine", "Secrétariat Général"), description, date de création. Un Workspace isole les affaires gérées par une unité spécifique.
- **Utilisateurs (Agents)** : Agent, Officier, Commissaire/Administrateur.
- **Actes (Mains Courantes, Plaintes, PV)** : Id, numéro d'enregistrement unique (généralisé automatiquement au format N°/ANNEE/COMMISSARIAT), type d'acte, date et heure des faits, déclaration/contenu textuel, quartier/arrondissement concerné, identité des parties (déclarant, victime, suspect s'il y a lieu)
- **Logs de Traçabilité (Audit Trail)**

### 4.2. API REST & Sécurité

Le serveur doit proposer des routes d'API strictes :

- POST /api/auth/login : Connexion sécurisée des fonctionnaires de police.
- GET /api/workspaces/:id/acts : Récupération filtrée des actes de la brigade concernée.
- POST /api/acts : Création d'une nouvelle plainte ou main courante avec génération du numéro officiel.
- GET /api/stats/criminality : Agrégation des données (nombre de plaintes par quartier, type

d'infraction le plus fréquent) pour générer des indicateurs statistiques.

## 5. Spécifications Fonctionnelles Détaillées (Interface Frontend)

| Module Fonctionnel                                          | Description des Fonctionnalités et Contraintes                                                                                                                                                                                                                                                                                                     |
|-------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>1. Tableau de Bord Analytique &amp; Stats</b>            | Interface principale présentant des graphiques simples ou des compteurs numériques (ex: Nombre de plaintes ce mois-ci, délits majeurs en cours, cartes par quartier/arrondissement). Permet d'avoir une vision globale de la criminalité en un coup d'œil.                                                                                         |
| <b>2. Module d'Enregistrement (Formulaires)</b>             | Formulaires de saisie guidés et fluides pour l'enregistrement rapide d'une main courante, d'une plainte ou d'une audition. Contrainte d'ergonomie : interface épurée, responsive (Mobile-First) permettant à un agent d'entrer des données depuis une tablette ou un smartphone lors d'une patrouille ou d'une intervention extérieure .           |
| <b>3. Moteur de Recherche Avancé &amp; Filtres</b>          | Recherche dynamique et instantanée par numéro d'acte, par nom de victime/suspect, ou par zone géographique (quartier) . Un bouton de filtre rapide permet d'isoler uniquement les dossiers "Urgents" ou "Non assignés" .                                                                                                                           |
| <b>4. Registre des Affaires Prioritaires (localStorage)</b> | L'officier de permanence peut "épingler" (icône d'étoile ou de ciblage) les enquêtes sensibles ou les mains courantes nécessitant une surveillance immédiate . Ces dossiers chauds sont stockés dans le localStorage du poste de l'agent pour être consultables instantanément au sommet de sa page de garde, même après fermeture du navigateur . |

## 6. Calendrier et Date de Livraison

Le projet est sectorisé selon le planning de livraison impératif suivant :

- **Lancement officiel du projet** : Jeudi 03 Juin 2026
- **Revue intermédiaire** : Vendredi 05 Juin 2026 via le premier dépôt GitHub (Structure de la base PostgreSQL et squelette de l'API REST)
- **Date Limite de Livraison** : Samedi 13 Juin 2026 à 13h59.

## 7. Livrables Attendus sur le Compte GitHub

La MOE doit structurer son dépôt de manière rigoureuse et professionnelle :

1. **Architecture propre des dossiers source** : Séparation nette entre /frontend (Vanilla HTML/CSS/JS) et /backend (API REST et scripts SQL d'initialisation de la base PostgreSQL) .
2. **Un fichier README.md professionnel** : Rédigé en Markdown, présentant l'application "SecurePost", son fonctionnement, les technologies employées, et la documentation pas-à-pas pour monter la base de données PostgreSQL en local.
3. **Historique des commits** : Un minimum de 10 commits réguliers, structurés et explicitement nommés pour valider la progression logique du travail.
4. **Déploiement en ligne** : Le Frontend doit être disponible publiquement (ex: GitHub Pages) et relié à une instance d'API et de base de données hébergée sur un cloud public (ex: Render, Railway) pour permettre le test en situation réelle sur smartphone par la MOA.
5. **Modélisation de données** : Schéma relationnel (MCD/MPD ou diagramme de tables) de la base de données PostgreSQL fourni au format startUML.
6. **Vidéo de démonstration** : Lien vers une vidéo YouTube enregistrée par l'équipe, illustrant le parcours complet d'un acte : de sa création sécurisée par un agent jusqu'à son apparition dans l'historique des logs de traçabilité et les statistiques globales.